

```

In [20]: import numpy as np
import pandas as pd

#Leer datos de un archivo csv
df = pd.read_csv('data_studentlist.csv',header = 'infer')
#pasamos la altura de cm a m
df['Height'] = df['Height'].apply(lambda x : x/100)

#promedio de la altura de los varones
HM = df.loc[(df['Gender'] == 'M'),'Height'].mean().round(2)

#promedio de la altura de las estudiantes
HF = df.loc[(df['Gender'] == 'F'),'Height'].mean().round(2)

#promedio del peso de los hombres
WM = df.loc[(df['Gender'] == 'M'),'Weight'].mean().round(2)

#promedio de la altura de las estudiantes
WF = df.loc[(df['Gender'] == 'F'),'Weight'].mean().round(2)

#altura maxima entre los varones
HM_max = df.loc[(df['Gender'] == 'M'), 'Height'].max()
HM_max = round(HM_max,2)

#altura minima entre las mujeres
HF_min = df.loc[(df['Gender'] == 'F'), 'Height'].min()
HF_min = round(HF_min,2)

#peso mas ligero entre los varones mayores a 1.75
WM1_min = df.loc[(df['Gender'] == 'M') & (df['Height'] > 1.75), 'Weight'].min()

#mayor peso entre las mujeres menores a 1.60m
WF1_max = df.loc[(df['Gender'] == 'F') & (df['Height'] < 1.60), 'Weight'].max()

# La nota promedio de los alumnos (hombres y mujeres) sin ausencias ('N')
NP = df.loc[(df['Absence'] == 'N'),'Grade'].mean().round(2)

# La nota promedio de los alumnos (hombres y mujeres) con ausencias ('Y')
YP = df.loc[(df['Absence'] == 'Y'),'Grade'].mean().round(2)

# Altura promedio de los alumnos (hombres y mujeres) con grupo sanguíneo 'A' o 'AB'
ABP = df.loc[(df['Bloodtype'] == 'A') | (df['Bloodtype'] == 'AB'),'Height'].mean()

#Altura promedio de los alumnos varones con grupo sanguíneo 'A' o 'AB'
ABPM = df.loc[(df['Gender'] == 'M') & ((df['Bloodtype'] == 'A') | (df['Bloodtype'] == 'AB')),'Height'].mean()

#Edad promedio de los alumnos (hombres y mujeres) con absentismo ('Y') cuya nota es
AP = df.loc[((df['Absence'] == 'Y') & (df['Grade'] >= 3)), 'Age'].mean().round()

print('El promedio de la altura de los varones es -> ',HM,'m')
print('El promedio de la altura de las mujeres es -> ',HF,'m')
print('El promedio del peso de los hombres es -> ',WM,'Kg')

```

```

print('El promedio del peso de las mujeres es -> ',WF,'kg')
print('la altura maxima de los varoes es ->',HM_max,'m')
print ('La aultra minima entre las mujeres es ->',HF_min,'m')
print('El peso mas liguereo entre los varones mayotes a 1.75 m es ->',WM1_min,'Kg')
print('el mayor peso entre las mujeres menore a 1.60m es ->',WF1_max,'Kg')
print('La nota promedio de los alumnos (hombres y mujeres) sin ausencias (N) es ->')
print('La nota promedio de los alumnos (hombres y mujeres) con ausencias (Y) es ->')
print('Altura promedio de los alumnos (hombres y mujeres) con grupo sanguíneo A o A')
print('Altura promedio de los alumnos varones con grupo sanguíneo A o AB -> ', ABPM)
print('Edad promedio de los alumnos (hombres y mujeres) con absentismo (Y) cuya not

```

```

El promedio de la altura de los varones es -> 1.72 m
El promedio de la altura de las mujeres es -> 1.67 m
El promedio del peso de los hombres es -> 68.5 Kg
El promedio del peso de las mujeres es -> 50.44 kg
la altura maxima de los varoes es -> 1.82 m
La aultra minima entre las mujeres es -> 1.55 m
El peso mas liguereo entre los varones mayotes a 1.75 m es -> 61.3 Kg
el mayor peso entre las mujeres menore a 1.60m es -> 45.3 Kg
La nota promedio de los alumnos (hombres y mujeres) sin ausencias (N) es -> 2.08
La nota promedio de los alumnos (hombres y mujeres) con ausencias (Y) es -> 3.0
Altura promedio de los alumnos (hombres y mujeres) con grupo sanguíneo A o AB es ->
1.73 m
Altura promedio de los alumnos varones con grupo sanguíneo A o AB -> 1.73 m
Edad promedio de los alumnos (hombres y mujeres) con absentismo (Y) cuya nota es may
or o igual a 3 -> 24.0 años

```

```

In [21]: import numpy as np
import pandas as pd

#Leer datos de una archivo csv
df1 = pd.read_csv('data_census.csv',header = 'infer')

#modifica el tipo de dato de la columna 'Population'
df1['Population'] = df1['Population'].apply(lambda x: x.replace(',','')).astype(int)

#Suma la población con ProvinceCode igual a 115 o 116
PT = df1.loc[((df1["Province Code"] == 115 )| (df1["Province Code"] == 116)), 'Popul
print('Población total con ProvinceCode igual a 115 o 116 es ->',PT,'\n')

#Población promedio de Las ciudades donde hay más hombres (GenderRatio > 1)
PP = df1.loc[(df1['Gender Ratio']> 1), 'Population'].mean().round(2)
print('Población promedio de las ciudades donde hay más hombres (GenderRatio > 1) e

#Lugares con más hombres (GenderRatio > 1) y menos de 2 personas por hogar
LP = df1.loc[(df1['Gender Ratio']>1)& (df1['Person\'s Household']<2)]
print('Lugares con más hombres (GenderRatio > 1) y menos de 2 personas por hogar\n')

#Ordena el DataFrame en orden ascendente de 'Households' y muestra los 10 primeros
df1.sort_values(by='Households', ascending=True, inplace=True)
print('DataFrame ordenado en orden ascendente deHouseholds:\n')
df1.head(10)

```

Población total con ProvinceCode igual a 115 o 116 es -> 5970327

Población promedio de las ciudades donde hay más hombres (GenderRatio > 1) es -> 419 221.21

Lugares con más hombres (GenderRatio > 1) y menos de 2 personas por hogar

	Province Code	City Code	Population	Households	Person's Household \
62	112	1155	20825	11,172	1.86
149	101	1240	26790	13,433	1.99
152	101	1017	30066	15,485	1.94
167	117	1033	38027	19,145	1.99
227	113	1120	43566	22,248	1.96
253	104	1163	10304	5,365	1.92

	Gender Ratio
62	1.29
149	1.27
152	1.15
167	1.03
227	1.11
253	1.15

DataFrame ordenado en orden ascendente deHouseholds:

Out[21]:

	Province Code	City Code	Population	Households	Person's Household	Gender Ratio
52	112	1001	2914271	1,145,232	2.54	1.01
228	104	1001	2697791	1,160,150	2.33	1.01
254	103	1001	3356540	1,356,430	2.47	1.02
26	108	1001	3517491	1,430,441	2.46	0.98
200	114	1179	23308	10,877	2.14	0.98
266	103	1010	252823	100,073	2.53	1.12
256	103	1201	258719	100,745	2.57	1.04
37	108	1042	248177	101,090	2.46	0.97
70	107	1064	242758	101,201	2.40	1.02
230	104	1226	246634	101,508	2.43	1.06

```
In [22]: import pandas as pd
import numpy as np

#Leer datos de un archivo csv
df2 = pd.read_csv('data_sales.csv', header='infer')

#Añade una nueva variable Amount = UnitPrice x Units y muestra las 5 primeras obser
df2['Amount'] = df2['Unit Price'] * df2['Units']
print("Tabla actualizada: \n",df2.head(5))
```

```
#Calcula el precio unitario promedio de cada región. Utiliza el método groupby()
Pr = df2.groupby('Region')['Unit Price'].mean().round(2)
print('\nPromedio de Unit Price usando groupby: \n',Pr)

#Calcula el precio unitario promedio de cada región. Utiliza el método pivot_table()
dfr = pd.pivot_table(df2,index = 'Region' ,values = 'Unit Price', aggfunc='mean').ro
print('\nPromedio de Unit Price usando pivot_table: \n',dfr.head())

'''
Calcula el total de unidades para cada región y tipo de artículo en una línea de código
Utiliza el método pivot_table() y rellena los valores faltantes con 0.
'''
dfr2 = pd.pivot_table(df2,index=['Region','Item'], values = 'Units' ,aggfunc='sum'
print('\nTotal de unidades para cada region:\n',dfr2)

'''
Calcula el Total de ventas para cada región y el tipo de artículo en una línea de código
Utiliza el método pivot_table() y rellena los valores faltantes con 0
'''
dfr3=pd.pivot_table(df2,index=['Region','Item'], values = 'Amount',aggfunc='sum' ,
print('\nTotal de ventas para cada region y tipo de articulo:\n')
dfr3.head(15)
```

Tabla actualizada:

	Date	Region	Item	Unit Price	Units	Amount
0	04-Jul-14	East	Pen Set	4.99	62	309.38
1	12-Jul-14	East	Binder	1.99	29	57.71
2	21-Jul-14	Central	Pen Set	12.49	55	686.95
3	29-Jul-14	East	Binder	19.99	81	1619.19
4	07-Aug-14	Central	Pen Set	23.95	42	1005.90

Promedio de Unit Price usando groupby:

Region

Central 18.02

East 9.14

West 53.66

Name: Unit Price, dtype: float64

Promedio de Unit Price usando pivot_table:

Unit Price

Region

Central 18.02

East 9.14

West 53.66

Total de unidades para cada region:

Units

Region Item

Central Binder 424

Desk 7

Pen 27

Pen Set 243

Pencil 498

East Binder 234

Pen 175

Pen Set 152

Pencil 130

West Binder 64

Desk 3

Pen 76

Pencil 88

Total de ventas para cada region y tipo de articulo:

Out[22]:

		Amount
Region	Item	
Central	Binder	5762.63
	Desk	875.00
	Pen	539.73
	Pen Set	2421.39
	Pencil	1540.32
East	Binder	2535.66
	Pen	1354.25
	Pen Set	1748.48
	Pencil	363.70
West	Binder	1279.36
	Desk	825.00
	Pen	151.24
	Pencil	231.12

In [26]:

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#Leer daros de un archivo csv
df3 = pd.read_csv('data_coffeeshop.csv', header='infer', na_values=[' '])
#ordenamos el data frame por año de inicio
#df3.sort_values(by='Year of Start', ascending=True, inplace=True)
#print(df3)

#tabla de frecuencias para la variable 'yearOfStart'
# y visualízala por año.
Frequencies = df3['Year of Start'].value_counts()
my_counts = list(Frequencies.values)
my_labels = list(Frequencies.index)
df4 = pd.DataFrame({'Size of Site':my_counts},index=my_labels)

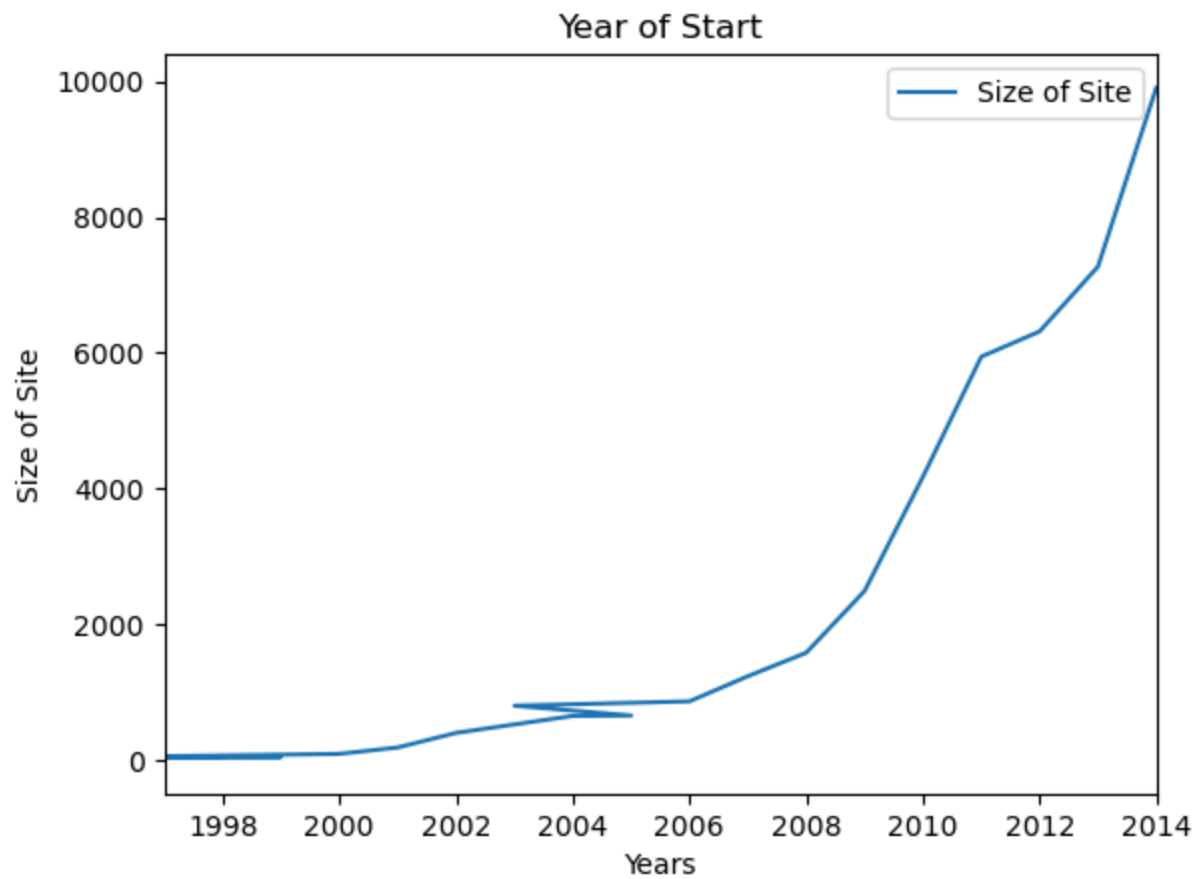
df4.plot()
plt.title('Year of Start')
plt.xlabel('Years')
plt.ylabel('Size of Site')
plt.xlim(1997,2014)

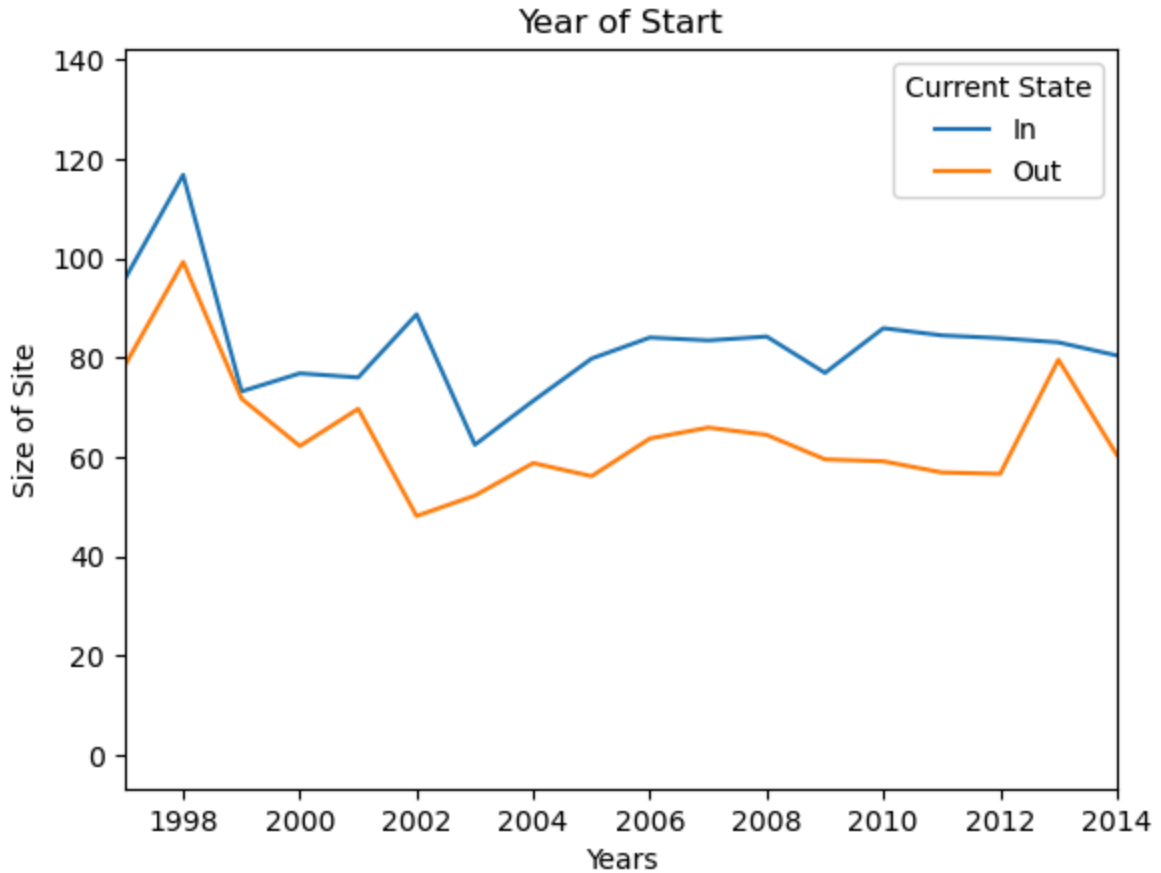
#Divide los datos por el estado actual del negocio ('In' o 'Out')
sr1 = pd.pivot_table(df3,index='Year of Start',columns='Current State',values='Size

```

```
sr1.plot()  
plt.title('Year of Start')  
plt.ylabel('Size of Site')  
plt.xlabel('Years')  
plt.xlim(1997, 2014)
```

Out[26]: (1997.0, 2014.0)





5. ¿Cuál es la sintaxis correcta para crear un arreglo de NumPy?
 - ☐ `np.createArray([1, 2, 3, 4, 5])`
 - ☒ `np.array([1, 2, 3, 4, 5])`
 - ☐ `np.object([1, 2, 3, 4, 5])`
6. ¿Cuál de las siguientes opciones es un arreglo de dos dimensiones?
 - ☐ 42
 - ☒ `[[1, 2, 3], [4, 5, 6]]`
 - ☐ `[1, 2, 3, 4, 5]`
7. ¿Cuál es la sintaxis correcta para comprobar el número de dimensiones de un arreglo?
 - ☐ `np.dim()`
 - ☐ `np.ndim()`
 - ☒ `np.ndim`
 - ☐ `np.dim`
8. ¿Cuál es la sintaxis correcta para imprimir el primer elemento de un arreglo?
 - ☒ `print(myArr[0])`
 - ☐ `print(myArr,1)`
 - ☐ `print(myArr[1])`
9. ¿Cuál es la sintaxis correcta para imprimir el número 8 de la siguiente matriz?


```
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
```

 - ☐ `print(arr[3, 0])`
 - ☐ `print(arr[7, 2])`
 - ☒ `print(arr[1, 2])`

10. ¿Cuál es la sintaxis correcta para imprimir los números [3, 4, 5] de la siguiente matriz?

```
arr = np.array([1,2,3,4,5,6,7])
```

- ☐ a `print(arr[2:4])`
- ☒ b `print(arr[2:5])`
- ☐ c `print(arr[2:6])`
- ☐ d `print(arr[3:6])`

11. ¿Cuál es el método correcto para unir dos o más arreglos?

- ☐ a `array_join()`
- ☒ b `concatenate()`
- ☐ c `join()`

12. ¿Cuál es la sintaxis correcta para devolver el índice de todos los elementos que tienen el valor 4 de la siguiente matriz?

```
arr = np.array([1,4,3,4,5,4,4])
```

- ☐ a `arr.search(4)`
- ☐ b `arr.where()`
- ☒ c `np.where(arr==4)`

13. Cuando se utiliza el módulo random de NumPy, ¿cómo se puede devolver un número aleatorio entre 0 y 100?

- ☒ a `random.rand(100)`
- ☐ b `random.rand()`
- ☐ c `random.randint(100)`

Samsung Innovation Campus

14. Cuando se utiliza el módulo random de NumPy, ¿cómo se puede devolver una Distribución Normal de Datos con 1000 números, concentrados alrededor del número 50 y con una desviación estándar de 0.2?

- ☒ a `random.normal(size=1000, loc=50, scale=0.2)`
- ☐ b `random.normal(size=1000, mean=50, deviation=0.2)`
- ☐ c `random.normal(size=1000, normal=50, s=0.1)`

15. ¿Cuál es la sintaxis correcta para sumar matemáticamente los números de arr1 a los números de arr2?

- ☒ a `np.add(arr1, arr2)`
- ☐ b `np.append(arr1, arr2)`
- ☐ c `sum(arr1, arr2)`

16. ¿Cuál es la sintaxis correcta para restar los números de arr1 de los números de arr2?

- ☐ a `np.min(arr1, arr2)`
- ☐ b `np.minus(arr1, arr2)`
- ☐ c `np.sub(arr1, arr2)`
- ☒ d `np.subtract(arr1, arr2)`

17. ¿Cuál es el método correcto para redondear decimales en NumPy?

- ☐ a `np.fix()`
- ☐ b `np.trunc()`
- ☐ c Las otras 3 opciones son métodos válidos en NumPy
- ☒ d `np.around()`

18. ¿Cuál sería la respuesta de esta suma acumulativa en NumPy?

```
arr = np.array([1,2,3])  
print(np.cumsum(arr))
```

- ☐ [3, 6, 9]
- ☐ [6]
- ☐ [9]
- ☒ [1 3 6]

19. ¿Cuál es la sintaxis correcta para crear una Series de pandas a partir de una lista?

- ☒ `pd.Series(mylist)`
- ☐ `pd.createSeries(mylist)`
- ☐ `pd.getSeries(mylist)`